Strategy Pattern with Generics

by Dr. Heinz M. Kabutz

Abstract:

The Strategy Pattern is elegant in its simplicity. With this pattern, we should try to convert intrinsic state to extrinsic, to allow sharing of strategy objects. It gets tricky when each strategy object needs a different set of information in order to do its work. In this newsletter, we look at how we can use Java 5 Generics to pass the correct subtype of the context into each strategy object.

Welcome to the 123rd edition of **The Java(tm) Specialists' Newsletter**, where we look at what happens when you combine the Strategy Pattern with Generics. This has been one of the most difficult and time-consuming newsletters to write, and I am grateful to the following people for providing valuable input: Angelika Langer, Philip Wadler, Maurice Naftalin and Kirk Pepperdine.

Before we delve into the details of this pattern, I need to add a disclaimer. You would typically use the Strategy Pattern for examples that are more complex than I am presenting here. There are other ways of solving this particular issue. However, in this newsletter I am showing you the steps that you would take to introduce the **Strategy Pattern** to existing code.

My goals are to:

- Move behaviour out of one big class.
- Make the algorithm for calculating tax exchangeable.
- Minimize number of objects by making state extrinsic.

This will help me to have code that I can extend without modifying what is there already.

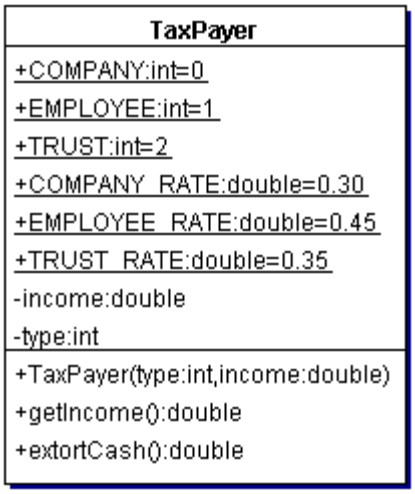
Strategy Pattern with Generics

In our Design Patterns Course [http://www.javaspecialists.eu/courses/dpc.jsp], one of the patterns that we look at in detail is the Strategy Pattern. It is a simple pattern, but has such a wide application that it is worth examining closely. The first part of the discussion involves converting existing code to this pattern, commonly recognised as switch or multi-conditional statements. The second part revolves around converting intrinsic state into extrinsic, to allow us to share strategy objects between different contexts.

We start with a simple class for calculating tax for different categories of TaxPayers. The type of TaxPayer is defined as an **`int`** inside the class, and we use that to decide how to calculate how much tax they must pay.

```
public class TaxPayer {
  public static final int COMPANY = 0;
  public static final int EMPLOYEE = 1;
  public static final int TRUST = 2;
  private static final double COMPANY_RATE = 0.30;
  private static final double EMPLOYEE_RATE = 0.45;
  private static final double TRUST_RATE = 0.35;

  private double income;
  private final int type;
  public TaxPayer(int type, double income) {
    this.type = type;
    this.income = income;
  }
  public double getIncome() {
    return income;
  }
  public double extortCash() {
    switch (type) {
      case COMPANY:  return income * COMPANY_RATE;
      case EMPLOYEE: return income * EMPLOYEE_RATE;
      case TRUST:    return income * TRUST_RATE;
      default:       throw new IllegalArgumentException();
    }
  }
}
```

Here is a simple UML class diagram of the large class:



This code is used from within the ReceiverOfRevenue, a type of IRS:

```
public class ReceiverOfRevenue {
  public static void main(String[] args) {
    TaxPayer heinz = new TaxPayer(TaxPayer.EMPLOYEE, 50000);
    TaxPayer maxsol = new TaxPayer(TaxPayer.COMPANY, 100000);
    TaxPayer family = new TaxPayer(TaxPayer.TRUST, 30000);
    System.out.println(heinz.extortCash());
    System.out.println(maxsol.extortCash());
    System.out.println(family.extortCash());
  }
}
```

The initial TaxPayer class is reasonable for a simple calculation. However, whenever the calculation changes, you need to modify the original class. In addition, if you have several **switch** statements in your class, you have to update every one when you introduce a new tax payer category. I once wrote a CASE tool using this approach. When I added new constructs, I had to change 5 different **switch** statements to accommodate the change. If I forgot the printing **switch** statement, then the new construct did not print.

A better approach is to avoid using a **switch** or multi-conditional if-else-if and instead use the Strategy pattern. This allows us to have a separate class for each tax calculation. We can then add new tax categories more easily. In addition, having smaller classes means that my unit testing becomes more focused on testing one particular situation.

To convert the original **switch** statement to a strategy, we first define the TaxStrategy interface:

```
public interface TaxStrategy {
  public double extortCash(double income);
}
```

We then have a separate strategy implementation for each calculation (as noted before, whilst it is simplistic now, it will become more complicated later):

```
public class CompanyTaxStrategy implements TaxStrategy {
  private static final double RATE = 0.30;
  public double extortCash(double income) {
    return income * RATE ;
  }
}

public class EmployeeTaxStrategy implements TaxStrategy {
  private static final double RATE = 0.45;
  public double extortCash(double income) {
    return income * RATE;
  }
}

public class TrustTaxStrategy implements TaxStrategy {
```

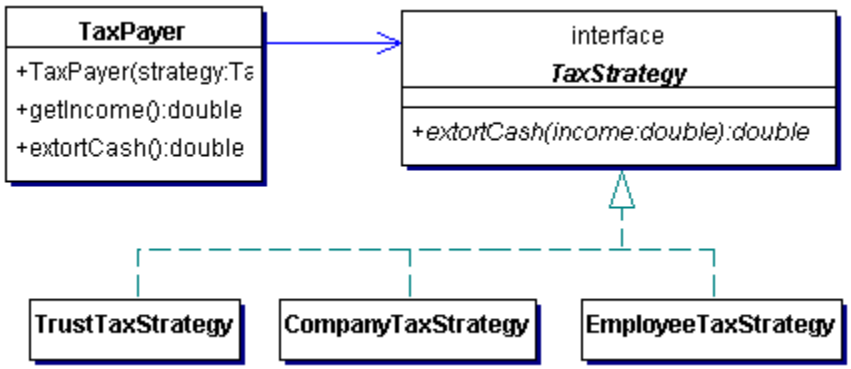
```
  private static final double RATE = 0.40;
  public double extortCash(double income) {
    return income * RATE;
  }
}
```

The TaxPayer class now has TaxStrategy objects instead of an **int** type, and we use Polymorphism instead of a multi-conditional statement. We change it to the following:

```
public class TaxPayer {
  public static final TaxStrategy EMPLOYEE =
    new EmployeeTaxStrategy();
  public static final TaxStrategy COMPANY =
    new CompanyTaxStrategy();
  public static final TaxStrategy TRUST =
    new TrustTaxStrategy();

  private final TaxStrategy strategy;
  private final double income;

  public TaxPayer(TaxStrategy strategy, double income) {
    this.strategy = strategy;
    this.income = income;
  }

  public double getIncome() {
    return income;
  }

  public double extortCash() {
    return strategy.extortCash(income);
  }
}
```

Because we have defined the types as constants within TaxPayer, we can keep the ReceiverOfRevenue class exactly as it was.

To illustrate this solution, please have a look at the class diagram:



There is one issue with this solution. The strategy's extortCash method is only given the TaxPayer's income. What if there are other factors that need to be taken into account for a certain TaxPayer? For example, when I got married, the tax law in South Africa had an absurd clause that married women paid more tax than unmarried women. (I therefore became the wife for tax purposes, since I was still a student and my wife the breadwinner ;-)

There are several solutions to this issue. One is to have each TaxStrategy instance associated with exactly one TaxPayer object. This means that the state is *intrinsic*, rather than *extrinsic*, and so we will not be able to share those TaxStrategy instances.

Another solution is to pass all variables through to the extortCash method. However, this is not that maintainable since you need to modify the method every time a new tax category is defined. Your government might declare a special tax dispensation for small companies.

Yet another solution is to pass a TaxPayer into the extortCash() method. If you then have an Employee subclass of TaxPayer, the

EmployeeTaxStrategy would need to *downcast* the instance to an Employee to find out whether this was a married woman or not.

The TaxStrategy interface would therefore change to:

```
public interface TaxStrategy {
  public double extortCash(TaxPayer p);
}
```

The extortCash() method in TaxPayer would change to:

```
public class TaxPayer {
  // the rest of the class is the same
  public double extortCash() {
    return strategy.extortCash(this);
  }
}
```

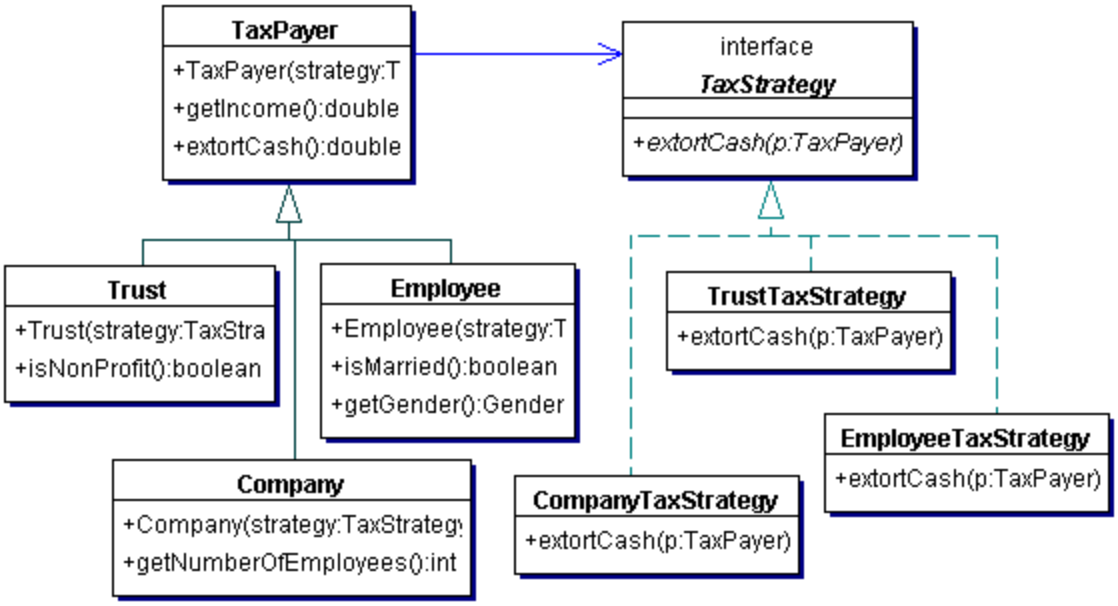
Let's take the strange example of a married woman paying more tax. We do not want TaxPayer to contain this detailed information, since it is information only relevant to employees. Companies might have other data that is important for calculating the tax. Trusts for non-profit might be zero rated.

```
public class Employee extends TaxPayer {
  public enum Gender { MALE, FEMALE }
  private final boolean married;
  private final Gender gender;

  public Employee(TaxStrategy strategy, double income,
        boolean married, Gender gender) {
    super(strategy, income);
    this.married = married;
    this.gender = gender;
  }

  public boolean isMarried() {
    return married;
  }

  public Gender getGender() {
    return gender;
  }
}
```

The EmployeeTaxStrategy now needs to find out whether this particular TaxPayer is a married female. To do that, it needs to downcast to an Employee. A casting mistake would only appear at runtime. We could check with **`instanceof`** whether it is an Employee before casting, but what do we do if it is **not** an Employee?

```
public class EmployeeTaxStrategy implements TaxStrategy {
  private static final double NORMAL_RATE = 0.40;
  private static final double MARRIED_FEMALE_RATE = 0.48;

  public double extortCash(TaxPayer p) {
    Employee e = (Employee) p; // here we need to downcast!!!
    if (e.isMarried() &&
        e.getGender() == Employee.Gender.FEMALE) {
      return e.getIncome() * MARRIED_FEMALE_RATE;
    }
    return e.getIncome() * NORMAL_RATE;
  }
}
```

The class diagram now looks like this:



This solution will work, but is not satisfactory. Let's look at another idea.

Avoiding Downcasting with Generics

Generics are supposed to solve the need for downcasting, right? Unfortunately, generics can also make code harder to read. If you are still struggling with the `<? super Object>` notation you would find our Java 5 Delta Course very useful - as we specifically cover generics in chapter 3 of the course.

I tried adding generics, but it was harder than I thought. After struggling for a while, I enlisted the help of Philip Wadler, Maurice Naftalin and Angelika Langer, a fellow Java Champion who has written an excellent Java Generics FAQ [http://www.angelikalanger.com/GenericsFAQ/JavaGenericsFAQ.html]. Together we worked out this solution.

We start by generifying TaxStrategy, so that it is bound to a subclass of TaxPayer.

```
public interface TaxStrategy<P extends TaxPayer> {
  public double extortCash(P p);
}
```

We can now write a CompanyTaxStrategy that is bound to the Company class. In other words, the parameter P is a company. In this country, a small company pays less tax, and is defined by having less than $1m income but more than 5 employees.

```
public class CompanyTaxStrategy implements TaxStrategy<Company> {
  private static final double BIG_COMPANY_RATE = 0.30;
  private static final double SMALL_COMPANY_RATE = 0.15;

  public double extortCash(Company company) {
    if (company.getNumberOfEmployees() > 5
        && company.getIncome() < 1000000) {
      return company.getIncome() * SMALL_COMPANY_RATE;
    }
    return company.getIncome() * BIG_COMPANY_RATE;
  }
}
```

We have to change the TaxPayer to contain the generic parameter `P`, which would be a subclass of TaxPayer. The expression `class TaxPayer<P extends TaxPayer<P>>` looks a bit strange and takes some getting used to. It is similar to the `Enum<E extends Enum<E>>` definition that confused Ken Arnold. After some thought, I think that `Enum<E extends Enum>` would have been sufficient. However, as Philip Wadler pointed out to me, since Enum is a generic class, we should only ever use it in conjunction with a type. In future, Java might become more strict and raw types could become illegal. We therefore have the choice of writing either `Enum<E extends Enum<E>>` or `Enum<E extends Enum<?>>`, of which the first option is more accurate. Even though the compiler currently does not show me a difference, I might see warnings in future, so I follow that idiom in my

class as well.

Inside TaxPayer, we want to use the derived class P directly in the extortCash() method. However, the only way of getting an instance without compiler warnings is to pass it in from the subclass. We can solve this issue with a factory method called `getDetailedType()` that returns the subclass.

```
public abstract class TaxPayer<P extends TaxPayer<P>> {
  public static final TaxStrategy<Employee> EMPLOYEE =
    new EmployeeTaxStrategy();
  public static final TaxStrategy<Company> COMPANY =
    new CompanyTaxStrategy();
  public static final TaxStrategy<Trust> TRUST =
    new TrustTaxStrategy();

  private double income;
  private TaxStrategy<P> strategy;

  public TaxPayer(TaxStrategy<P> strategy, double income) {
    this.strategy = strategy;
    this.income = income;
  }

  protected abstract P getDetailedType();

  public double getIncome() {
    return income;
  }

  public double extortCash() {
    return strategy.extortCash(getDetailedType());
  }
}
```

When we write our Employee, we specify that it must only be created with a TaxStrategy for Employees. Any other TaxStrategy will cause compiler warnings, rightfully so.

```
public class Employee extends TaxPayer<Employee> {
  public enum Gender { MALE, FEMALE }
  private final boolean married;
  private final Gender gender;
  public Employee(TaxStrategy<Employee> strategy, double income,
                  boolean married, Gender gender) {
    super(strategy, income);
    this.married = married;
    this.gender = gender;
  }

  protected Employee getDetailedType() {
    return this;
  }

  public boolean isMarried() {
    return married;
  }

  public Gender getGender() {
    return gender;
  }
}
```

The beauty of using generics here is that EmployeeTaxStrategy does not need to downcast anymore. It is tightly bound to Employees through the generic type.

```
public class EmployeeTaxStrategy implements TaxStrategy<Employee> {
  private static final double NORMAL_RATE = 0.40;
  private static final double MARRIED_FEMALE_RATE = 0.48;
```

```
  public double extortCash(Employee e) {
    if (e.isMarried() &&
        e.getGender() == Employee.Gender.FEMALE) {
      return e.getIncome() * MARRIED_FEMALE_RATE;
    }
    return e.getIncome() * NORMAL_RATE;
  }
}
```

You can have different strategies for employees, but the EmployeeTaxStrategy can only be used for employees. This is enforced by the compiler.

It is amazing how quickly Generics start to seem obvious, then phase into obscurity just as fast.